

2、核心组件之模型

1、模型初始化

1.1、直接创建模型对象

1.2、使用init_chat_model

2、模型调用

1、invoke 和 stream

2、batch

3、batch_as_completed

3、模型结果结构化

1、使用 json Schme 来规范输出格式

2、使用pydantic

1、基本案例

2、嵌套输出

3、使用 TypedDict 来控制输出结果

4、参数include_raw=True

4、Agent 接入模型

1、静态模型

2、动态模型

大型语言模型（LLM）是强大的 AI 工具，能够像人类一样解释和生成文本。它们功能多样，可以撰写内容、翻译语言、总结和回答问题，而无需针对每个任务进行专门训练。除了文本生成，许多模型还支持：

- **工具调用** – 调用外部工具（如数据库查询或 API 调用）并在其响应中使用结果。
- **结构化输出** – 模型响应被限制为遵循定义的格式。
- **多模态** – 处理和返回除文本之外的数据，例如图像、音频和视频。
- **推理** – 模型执行多步推理以得出结论。

模型是智能体的推理引擎。它们驱动代理的决策过程，确定调用哪些工具、如何解释结果以及何时提供最终答案。您选择的模型的质量和直接影响代理的可靠性和性能。不同的模型擅长不同的

任务——有些更擅长遵循复杂指令，有些更擅长结构化推理，有些支持更大的上下文窗口来处理更多信息。

1、模型初始化

1.1、直接创建模型对象

比如使用 `langchain_openai` 中的 `ChatOpenAI`

```
Python |
1  from langchain_openai import ChatOpenAI
2  # 创建一个和调用大模型的对象
3  model = ChatOpenAI(
4      model=os.getenv("llm_model"),
5      base_url=os.getenv("base_url"),
6      api_key=os.getenv("api_key"),
7  )
```

1.2、使用 `init_chat_model`

`init_chat_model` 是 langchain 中封装的一个模型初始化方法，可用于快速对模型进行初始化，其内部还是调用相关的第三方库来初始化模型,也需要安装对应大模型接入的模型

```
Python |
1  from langchain.chat_models import init_chat_model
2
3
4  model = init_chat_model(
5      model=os.getenv("llm_model"),
6      model_provider="openai",
7      base_url=os.getenv("base_url"),
8      api_key=os.getenv("api_key"),)
```

2、模型调用

1、invoke 和 stream

这两种调用方式前面有做讲解，本节课不再介绍

```
Python |  
1 # invoke  
2 response = model.invoke("你是谁")  
3 print(response)  
4  
5 # stream  
6 for chunk in model.stream("Why do parrots have colorful feathers?"):  
7     print(chunk.text, end="|", flush=True)
```

2、batch

批量处理对模型的独立请求集合可以显著提高性能，因为处理可以并行完成。

`batch()` 将仅返回整个批次的最终输出

```
Python |  
1 responses = model.batch([  
2     "你是谁",  
3     "今天是几月几日",  
4 ])  
5  
6 for response in responses:  
7     print(response)
```

3、batch_as_completed

流式批量执行使用 `batch_as_completed`

```
Python |  
1 for response in model.batch_as_completed([  
2     "你是谁",  
3     "今天是几月几日",  
4 ]):  
5     print(response)
```

3、模型结果结构化

我们可以要求模型以符合给定模式的格式提供其响应。这对于确保输出能够轻松解析并在后续处理中使用非常有用。LangChain 支持多种模式类型和强制结构化输出的方法。

没有结构化输出的都是自然语言

```

1  # @Author : 木森
2  # @weixin: python771
3  import dotenv
4  import os
5  from langchain_openai import ChatOpenAI
6
7  # 把.env文件中的配置加载到环境变量中
8  dotenv.load_dotenv()
9
10 # 创建一个和调用大模型的对象
11 llm = ChatOpenAI(
12     model=os.getenv("llm_model"),
13     base_url=os.getenv("base_url"),
14     api_key=os.getenv("api_key"),
15 )
16
17 message = [
18     {"role": "user", "content": """
19     请根据以下需求文档,生成3条正向的测试用例
20     📌 F1.3 用户信息修改
21     🧩 功能背景
22         用户可修改昵称、密码、头像、性别等基础信息。
23     👤 主流程
24         1. 用户进入“个人中心”
25         2. 修改某字段并保存
26         3. 系统校验内容合法性 (如昵称长度、头像格式)
27         4. 修改成功后刷新显示
28     ⚠️ 异常流程
29         用户未登录: 提示登录后操作
30         输入非法字符: 提示不符合规范
31     📌 业务规则
32         昵称长度大于3 小于20, 支持中英文
33         性别只能为“男 / 女 / 保密”
34         头像图片限制大小 (2M以内), 格式为 png/jpg/jpeg
35
36     """}
37 ]
38
39 result = llm.invoke(message)
40 print(result.content)

```

比如我要生成如下格式的用例数据

```
1  [
2    {
3      "id": "用例编号"
4      "name": "测试用例名称",
5      "steps": [
6        "步骤描述1",
7        "步骤描述2"
8      ],
9      "expected": [
10       "预期结果断言1",
11       "预期结果断言2"
12     ],
13   }
14 ]
```

1、使用 json Schme 来规范输出格式

```
1 from typing import List, TypedDict
2 import dotenv
3 import os
4 from langchain_openai import ChatOpenAI
5 import json
6
7 # 把.env文件中的配置加载到环境变量中
8 dotenv.load_dotenv()
9
10 # 创建一个和调用大模型的对象
11 model = ChatOpenAI(
12     model=os.getenv("llm_model"),
13     base_url=os.getenv("base_url"),
14     api_key=os.getenv("api_key"),
15 )
16
17 message = [
18     {"role": "user", "content": """
19     请根据以下需求文档,生成1条正向的测试用例
20     📌 F1.3 用户信息修改
21     🧩 功能背景
22         用户可修改昵称、密码、头像、性别等基础信息。
23     👤 主流程
24         1. 用户进入"个人中心"
25         2. 修改某字段并保存
26         3. 系统校验内容合法性 (如昵称长度、头像格式)
27         4. 修改成功后刷新显示
28     ⚠️ 异常流程
29         用户未登录: 提示登录后操作
30         输入非法字符: 提示不符合规范
31     📌 业务规则
32         昵称长度大于3 小于20, 支持中英文
33         性别只能为"男 / 女 / 保密"
34         头像图片限制大小 (2M以内), 格式为 png/jpg/jpeg
35
36     """}
37 ]
38
39
40 # 定义 JSON Schema 来规范输出格式
41 json_schema = {
42     "title": "TestCase",
43     "description": "测试用例",
44     "type": "object",
45     "properties": {
```

```

46         "id": {
47             "type": "string",
48             "description": "用例编号"
49         },
50         "name": {
51             "type": "string",
52             "description": "测试用例名称"
53         },
54         "steps": {
55             "type": "array",
56             "items": {
57                 "type": "string"
58             },
59             "description": "测试步骤列表"
60         },
61         "expected": {
62             "type": "array",
63             "items": {
64                 "type": "string"
65             },
66             "description": "预期结果断言列表"
67         }
68     },
69     "required": ["id", "name", "steps", "expected"]
70 }
71
72 # 配置模型使用 JSON Schema 结构化输出
73 model_with_structure = model.with_structured_output(
74     json_schema,
75     method="json_schema",
76 )
77
78 # 调用模型
79 result = model_with_structure.invoke(message)
80 print("类型:", type(result))
81 print("结果:", result)

```

2、使用pydantic

1、基本案例

- 使用 pydantic 指定大模型输入结果的数据格式


```
1  # @Author : 木森
2  # @weixin: python771
3  from typing import List
4
5  import dotenv
6  import os
7  from langchain_openai import ChatOpenAI
8
9  # 把.env文件中的配置加载到环境变量中
10 dotenv.load_dotenv()
11
12 # 创建一个和调用大模型的对象
13 model = ChatOpenAI(
14     model=os.getenv("llm_model"),
15     base_url=os.getenv("base_url"),
16     api_key=os.getenv("api_key"),
17 )
18
19 message = [
20     {"role": "user", "content": ""
21      请根据以下需求文档,生成3条正向的测试用例
22      📌 F1.3 用户信息修改
23      🧩 功能背景
24          用户可修改昵称、密码、头像、性别等基础信息。
25      👤 主流程
26          1. 用户进入“个人中心”
27          2. 修改某字段并保存
28          3. 系统校验内容合法性 (如昵称长度、头像格式)
29          4. 修改成功后刷新显示
30      ⚠️ 异常流程
31          用户未登录: 提示登录后操作
32          输入非法字符: 提示不符合规范
33      📌 业务规则
34          昵称长度大于3 小于20, 支持中英文
35          性别只能为“男 / 女 / 保密”
36          头像图片限制大小 (2M以内), 格式为 png/jpg/jpeg
37
38     """]}
39 ]
40 # 导入Pydantic库, 用于数据验证和设置管理
41 from pydantic import BaseModel, Field
42
43
44 # 定义Movie数据模型类, 继承自BaseModel
45 class CaseItem(BaseModel):
```

```

46     id: str = Field(..., description="用例编号")
47     name: str = Field(..., description="测试用例名称")
48     steps: List[str] = Field(..., description="测试步骤列表")
49     expected: List[str] = Field(..., description="预期结果断言列表")
50
51
52
53 # 告诉语言模型要按照CaseItem类的结构来格式化输出
54 model_with_structure = model.with_structured_output(CaseItem)
55
56 result = model_with_structure.invoke(message)
57 print(type(result), result.model_dump())
58

```

2、嵌套输出

如果生成多条数据或者数据中有字段嵌套，可以使用 pydantic 模型类定义字段时指定字段嵌套来实现

```

1 # 导入Pydantic库，用于数据验证和设置管理
2 from pydantic import BaseModel, Field
3
4
5 # 定义Movie数据模型类，继承自BaseModel
6 class CaseItem(BaseModel):
7     id: str = Field(..., description="用例编号")
8     name: str = Field(..., description="测试用例名称")
9     steps: List[str] = Field(..., description="测试步骤列表")
10    expected: List[str] = Field(..., description="预期结果断言列表")
11
12 class CaseList(BaseModel):
13     cases: List[CaseItem]
14
15
16 # 告诉语言模型要按照CaseItem类的结构来格式化输出
17 model_with_structure = model.with_structured_output(CaseList)
18
19 result = model_with_structure.invoke(message)
20 print(type(result), result.model_dump())

```

3、使用 TypedDict 来控制输出结果

不推荐使用方法

```
1  # @Author   : 木森
2  # @weixin: python771
3  from typing import List
4
5  import dotenv
6  import os
7  from langchain_openai import ChatOpenAI
8
9  # 把.env文件中的配置加载到环境变量中
10 dotenv.load_dotenv()
11
12 # 创建一个和调用大模型的对象
13 model = ChatOpenAI(
14     model=os.getenv("llm_model"),
15     base_url=os.getenv("base_url"),
16     api_key=os.getenv("api_key"),
17 )
18
19 message = [
20     {"role": "user", "content": ""
21      请根据以下需求文档,生成3条正向的测试用例
22      📌 F1.3 用户信息修改
23      🧩 功能背景
24          用户可修改昵称、密码、头像、性别等基础信息。
25      👤 主流程
26          1. 用户进入“个人中心”
27          2. 修改某字段并保存
28          3. 系统校验内容合法性（如昵称长度、头像格式）
29          4. 修改成功后刷新显示
30      ⚠️ 异常流程
31          用户未登录：提示登录后操作
32          输入非法字符：提示不符合规范
33      📌 业务规则
34          昵称长度大于3 小于20，支持中英文
35          性别只能为“男 / 女 / 保密”
36          头像图片限制大小（2M以内），格式为 png/jpg/jpeg
37
38     """]}
39 ]
40
41
42
43 from typing import List, TypedDict
44
45
```

```

46 # 定义Movie数据模型类，继承自BaseModel
47 class CaseItem(TypedDict):
48     id: str
49     name: str
50     setup: str
51     expected: List
52
53
54 class CaseList(TypedDict):
55     cases: List[CaseItem]
56
57
58 # 告诉语言模型要按照CaseItem类的结构来格式化输出
59 model_with_structure = model.with_structured_output(CaseList)
60
61 result = model_with_structure.invoke(message)
62 print(type(result), result)
63

```

注意事项:

- TypedDict 只提供类型提示，没有数据验证功能
- TypedDict性能更好，但功能比 Pydantic 简单
- 适合不需要复杂验证的场景

4、参数 `include_raw=True`

加上参数 `include_raw=True`，大模型在输出结果时，不仅会返回格式化之后的结果，也会返回大模型输出的原始结果和元数据

```

1 # 告诉语言模型要按照CaseItem类的结构来格式化输出
2 model_with_structure = model.with_structured_output(CaseList, include_raw=True)
3
4 result = model_with_structure.invoke(message)
5 print(type(result), result.model_dump())

```

4、Agent 接入模型

1、静态模型

静态模型在创建智能体时配置一次，并在整个执行过程中保持不变。这是最常见和直接的方法。要从模型标识符字符串初始化静态模型：

```
Python |  
1 from langchain.agents import create_agent  
2  
3 agent = create_agent(  
4     "gpt-5",  
5     tools=tools  
6 )  
7  
8 agent = create_agent(  
9     model=model,  
10    tools=tools  
11 )
```

为了更好地控制模型配置，请直接使用提供者包初始化模型实例。

在示例中，使用 [ChatOpenAI](#)

```
Python |  
1 from langchain.agents import create_agent  
2 from langchain_openai import ChatOpenAI  
3  
4 model = ChatOpenAI(  
5     model="gpt-5",  
6     temperature=0.1,  
7     max_tokens=1000,  
8     timeout=30  
9     # ... (other params)  
10 )  
11 agent = create_agent(model, tools=tools)
```

模型实例让您完全可以完全控制配置。当您需要设置特定的参数时使用它们，例如

`temperature`、`max_tokens`、`timeouts`、`base_url` 和其他特定于提供者的设置。

2、动态模型

动态模型在运行时根据当前状态和上下文选择。这支持复杂的路由逻辑和成本优化。要使用动态模型，请使用 `@wrap_model_call` 装饰器创建中间件，该中间件会修改请求中的模型：

使用动态模型的优点：

- 轻松集成新模态（图片、音频、视频），多模态的支持更方便
- 简单对话使用成本较低的模型、复杂任务才启用高性能模型、降低成本
- 模块化扩展

```
1 from langchain_openai import ChatOpenAI
2 from langchain.agents import create_agent
3 from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
4
5 # 基础对话模型
6 basic_model = ChatOpenAI(
7     model=os.getenv("llm_model"),
8     base_url=os.getenv("base_url"),
9     api_key=os.getenv("api_key"),
10 )
11
12 # 图片理解模型
13 image_model = ChatOpenAI(
14     model=os.getenv("image_model"),
15     base_url=os.getenv("base_url"),
16     api_key=os.getenv("api_key"),
17 )
18
19 @wrap_model_call
20 def dynamic_model_selection(request: ModelRequest, handler) -> ModelResponse:
21     """基于对话复杂度选择模型"""
22     message_count = len(request.state["messages"])
23
24     # 根据消息数量选择模型
25     if message_count > 10:
26         selected_model = image_model
27     else:
28         selected_model = basic_model
29
30     request.model = selected_model
31     return handler(request)
32
33 # 创建带中间件的agent
34 agent = create_agent(
35     model=basic_model, # 默认模型
36     tools=tools,
37     # 配置中间件
38     middleware=[dynamic_model_selection]
39 )
```


